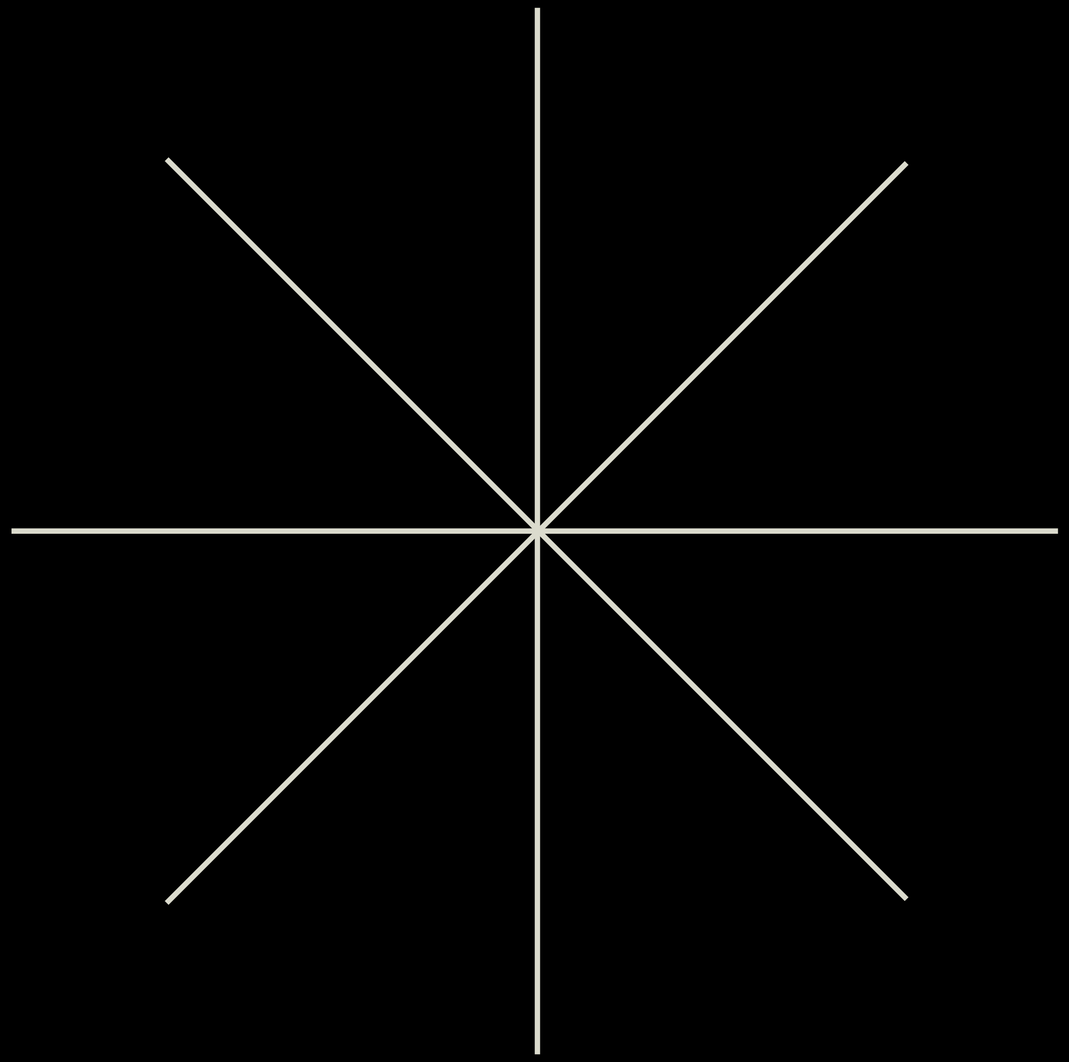
PUCPR - ESCOLA POLITÉCNICA
ENGENHARIA DE COMPUTAÇÃO
CONCEPÇÃO E DESIGN DE PROJETO TRANSFORMADOR



PLANO DE PROJETO

DETECÇÃO DE VULNERABILIDADES EM CÓDIGO-FONTE COM MACHINE LEARNING

ANDRÉ F. WOZNIACK & RICARDO H. PAMPLONA

ORIENTADOR: PROF. JULIO CESAR NIEVOLA

MOTIVAÇÃO

~45%

do código gerado por LLMs contém vulnerabilidades de segurança

VERACODE, 2025

PROBLEMA

As ferramentas de análise estática (SAST) dependem de regras predefinidas: não entendem o contexto do código e geram muitos falsos positivos.

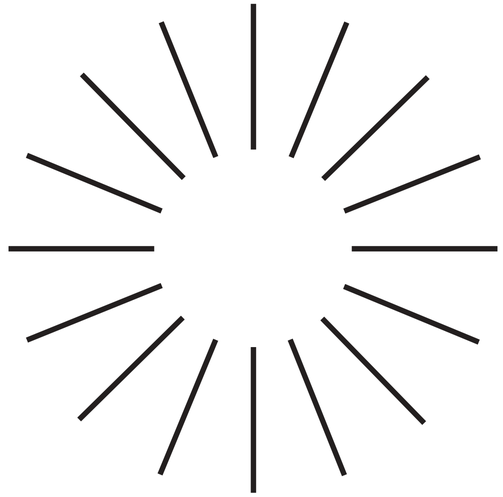
CONTEXTO

Assistentes de IA como Copilot, Claude Code e Codex aceleraram a produção de código. E todo código, humano ou de IA, precisa de verificação de segurança.

RELEVÂNCIA

Analisar o código antes do merge reduz o risco de exploração de falhas em produção.

OBJETIVO DO PROJETO



OBJETIVO GERAL

Desenvolver um software que analisa o código antes do merge e detecta vulnerabilidades combinando análise estática e Machine Learning.

RESULTADO ESPERADO

Protótipo funcional que reduz os falsos positivos encaminhados ao desenvolvedor e ajuda a priorizar os alertas de segurança.

ESCOPO INICIAL

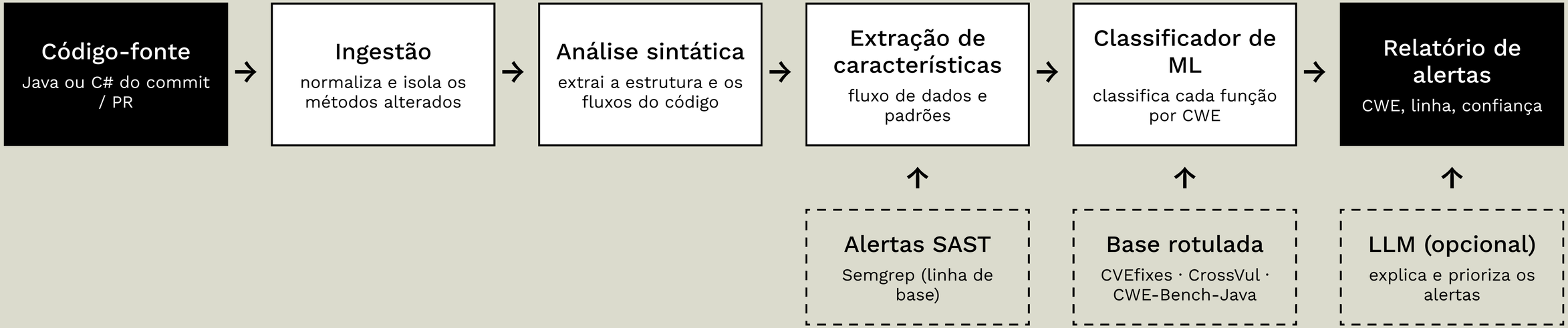
Java e C# (Java como prioridade do MVP) · CWE-22, 78, 79 e 89 (Path Traversal, Command Injection, XSS e SQL Injection).

VISÃO GERAL DA SOLUÇÃO

SOLUÇÃO 100% SOFTWARE

FLUXO PRINCIPAL (MVP)

TRACEJADO = APOIO / OPCIONAL



ENTRADA

Arquivos Java ou C# modificados em um commit ou pull request

PROCESSAMENTO

Extrai a estrutura e o fluxo de dados do código e classifica cada função com ML

SAÍDA

Relatório com a CWE, arquivo e linha, nível de confiança e recomendação de correção

TEORIAS E TÉCNICAS

01 · TEORIA

Análise estática e de taint

Analisar o código sem executá-lo, rastreando dados não confiáveis (source) até operações sensíveis (sink), como uma consulta SQL.

02 · ALGORITMOS

Classificador de referência

Um modelo simples e de baixo custo classifica cada função como vulnerável ou não vulnerável. O algoritmo será escolhido nas provas de conceito.

03 · PARADIGMAS

Aprendizado supervisionado + pipeline modular

O modelo aprende com exemplos de código vulnerável e corrigido; cada etapa do sistema é um módulo independente e testável.

04 · TECNOLOGIAS

Representações estruturadas do código

Árvores e grafos que descrevem a estrutura, a execução e o fluxo de dados do código. Code Property Graph, modelos pré-treinados e LLMs são extensões em avaliação.

HARDWARE E SOFTWARE

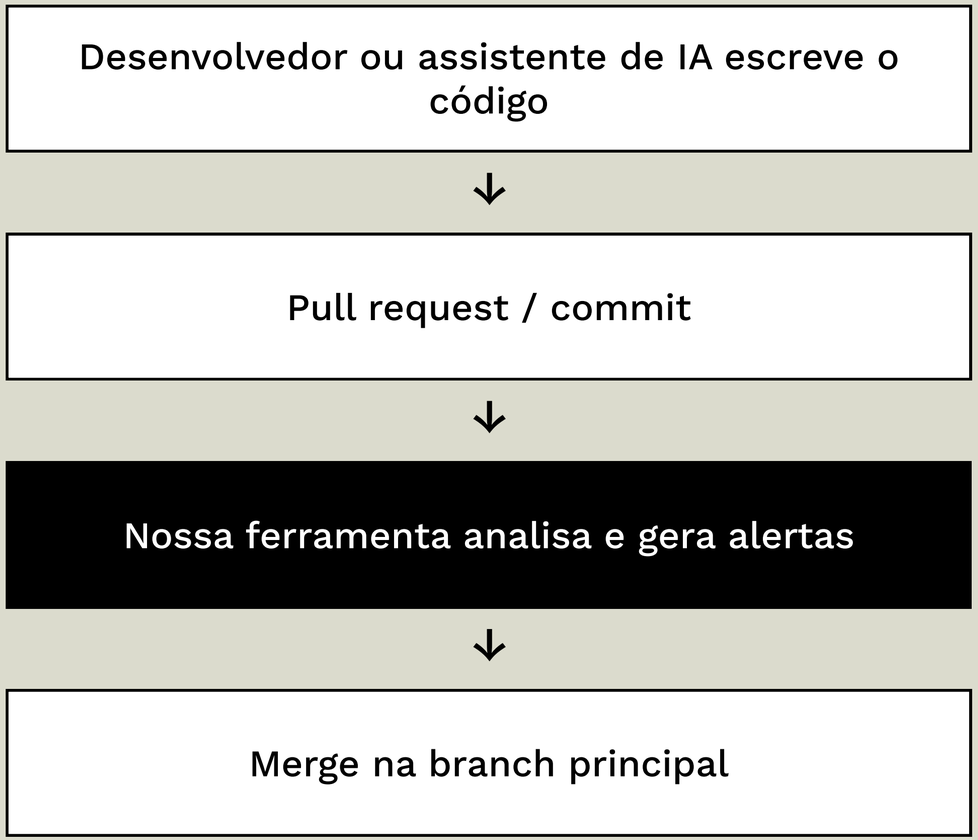
	RECURSO	PARA QUÊ	ORIGEM
HARDWARE	**PCs da equipe**	Desenvolvimento e testes do pipeline	PRÓPRIO
	GPU em nuvem	Colab ou Kaggle, só se o treino do modelo exigir	EXTERNO
SOFTWARE	**Python + scikit-learn**	Pipeline e classificadores de ML (linguagem em avaliação)	EXTERNO
	Semgrep + Joern	Análise estática (linha de base) e grafos de código (em avaliação)	EXTERNO
	Docker + Git	Ambiente reprodutível e versionamento	EXTERNO
DADOS	**Bases públicas**	CVEfixes, CrossVul e CWE-Bench-Java para treino e avaliação	EXTERNO

CONTEXTO

Público-alvo: equipes de desenvolvimento que revisam código antes do merge — inclusive código escrito com IA.

Aplicação: análise de commits e pull requests em projetos Java e C#, por linha de comando e integrável a pipelines de integração contínua.

ONDE A FERRAMENTA ATUA



REFERÊNCIAS PRINCIPAIS

PROBLEMA

PEARCE, H. et al. Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. IEEE S&P, 2022.

VERACODE. 2025 GenAI Code Security Report. 2025.

IRANMANESH, M. et al. ZeroFalse: Improving Precision in Static Analysis with LLMs. arXiv, 2025.

TÉCNICAS

YAMAGUCHI, F. et al. Modeling and discovering vulnerabilities with code property graphs. IEEE S&P, 2014.

ZHOU, Y. et al. Devign: effective vulnerability identification via graph neural networks. NeurIPS, 2019.

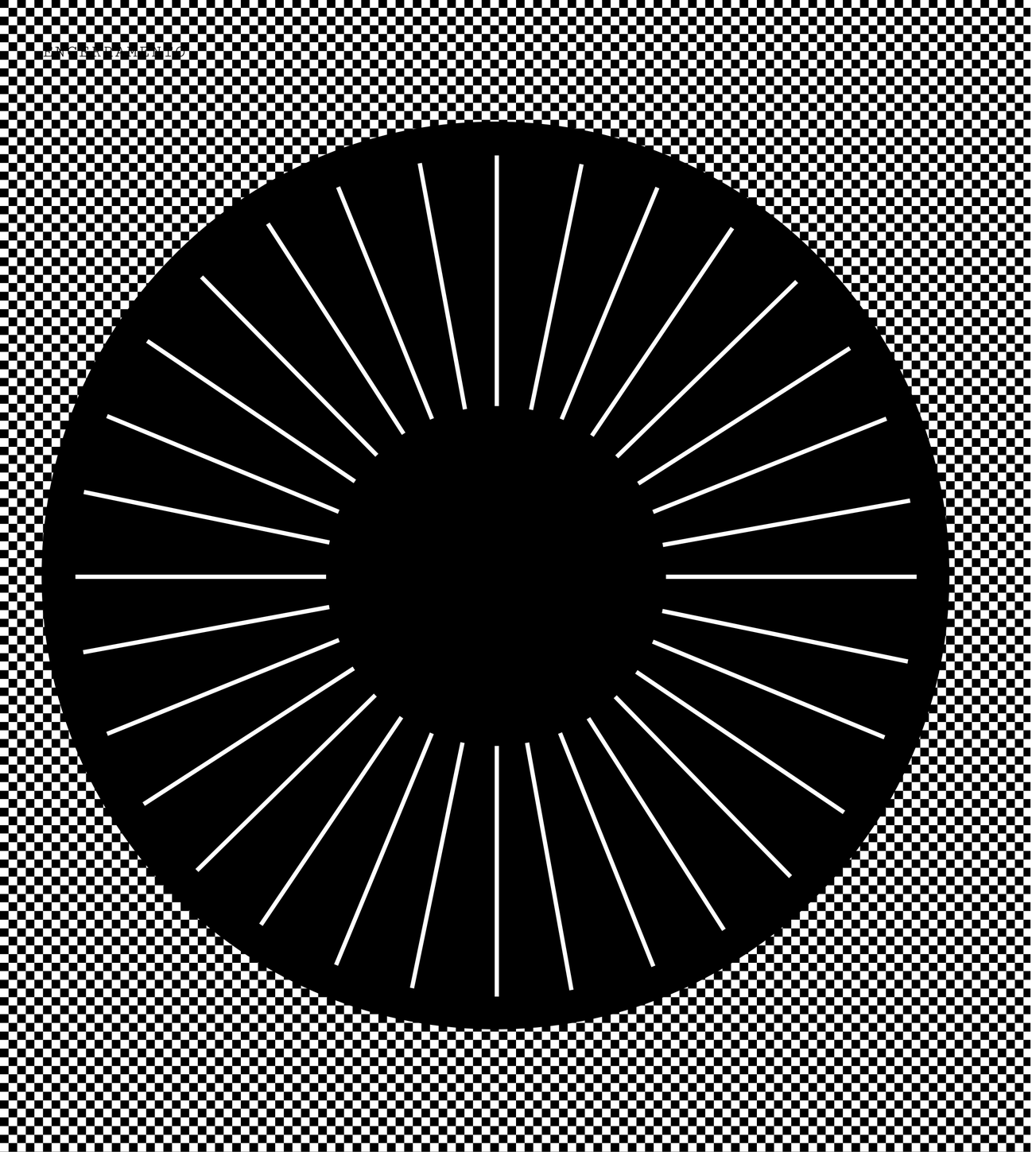
FENG, Z. et al. CodeBERT: a pre-trained model for programming and natural languages. EMNLP, 2020.

DADOS E AVALIAÇÃO

DING, Y. et al. Vulnerability Detection with Code Language Models: How Far Are We? arXiv, 2024.

CHAKRABORTY, S. et al. Deep learning based vulnerability detection: are we there yet? IEEE TSE, 2022.

LISTA COMPLETA NO DOCUMENTO DO PLANO DE PROJETO



EM RESUMO

DETECÇÃO DE VULNERABILIDADES NO CÓDIGO ANTES DO MERGE, COMBINANDO ANÁLISE ESTÁTICA E MACHINE LEARNING.

PRÓXIMOS PASSOS

OUT–NOV/26 · PROJETO FÍSICO

ATÉ ABR/27 · PROTÓTIPO

JUN/27 · DEFESA FINAL

OBRIGADO!